

Applied Machine Learning

Notes for Final Exam

1. Introduction to Machine Learning

Machine learning (ML) is a subfield of Artificial Intelligence (AI) that empowers systems to learn and improve from experience without explicit programming. Unlike traditional programming, where a specific program is written to produce an output from data, machine learning involves feeding data to a generic algorithm, which then builds its own logic or model to make predictions. This process is often described as "reverse engineering" a function from data.

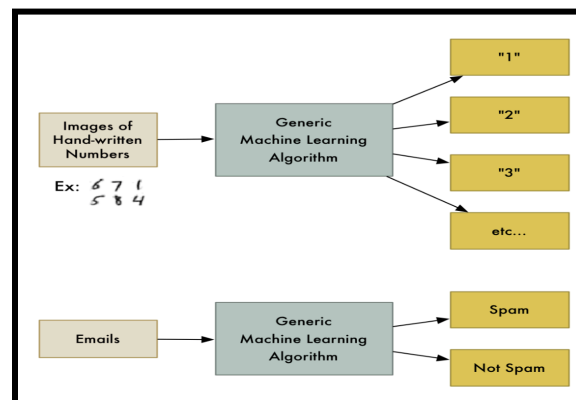
1.1. Comparison: Programming vs. Machine Learning

- **Traditional Programming:** Program + Data → Output (e.g., `def predictFahrenheit(Cel): return (1.8 * Cel + 32)`)
- **Machine Learning:** Data + Output → Program/Model (e.g., given Celsius and Fahrenheit values, determine the relationship/function)

Example: Fahrenheit to Celsius Conversion

Given a set of Celsius and corresponding Fahrenheit temperatures, machine learning (specifically, linear regression) can be used to predict the underlying function `def predictFahrenheit(Cel): return (1.8 * Cel + 32)`.

1.2. Applications of Machine Learning



Machine learning is an umbrella term for generic algorithms applied across various fields:

- **Internet and Cloud:** Image classification, speech recognition, language translation, text classification, sentiment analysis, recommendation systems.
- **Medicine and Biology:** Cancer cell detection, diabetic grading, drug discovery.

- **Security and Defense:** Face detection, video surveillance, satellite imagery.
- **Autonomous Machines:** Pedestrian detection, lane tracking, recognizing traffic signs.

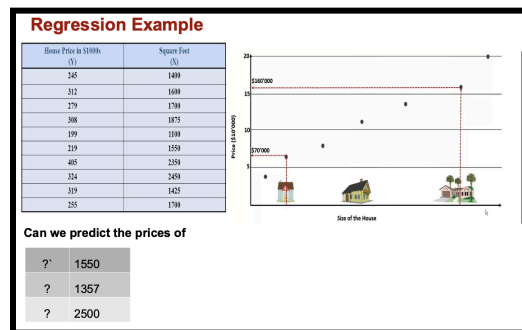
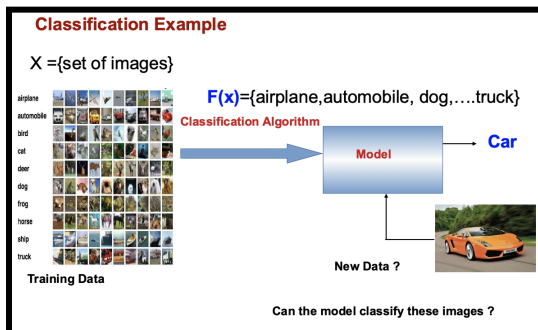
1.3. Types of Learning

Machine learning algorithms are broadly categorized based on the nature of the training data.

Supervised Learning

This paradigm involves training data that includes desired outputs, or labels. The goal is to **predict** a function $F(X)$ for new inputs X , **given** examples of input X and their corresponding output $Y=F(X)$.

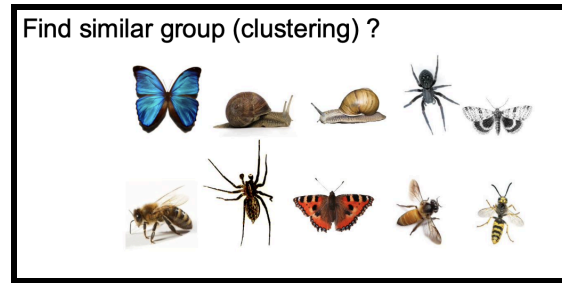
- **Classification:** Deals with discrete outputs (categories).
 - **Example:** Detecting spam emails (spam/not spam).
 - **Image Classification:** Given a set of images X , find $F(X)$ to classify them as "dog" or "cat". A model can be trained on labeled images (e.g., airplane, automobile, dog, truck) and then used to classify new, unseen images (e.g., identifying a new image as a "cat" or "car").
- **Regression:** Handles continuous outputs (values).
 - **Example:** Predicting stock prices within a specific timeframe.
 - **House Price Prediction:** Given a dataset of house prices and their corresponding square footage, a regression model can predict the price of a new house based on its square footage.



Unsupervised Learning

This approach works with unlabeled input data. The algorithm's objective is to infer hidden structures or patterns within the data. It is widely used in pattern recognition, image analysis, information retrieval, bioinformatics, and data compression.

- **Clustering:** A common unsupervised technique that involves grouping similar data points together based on similarities.
 - **Example:** Grouping similar insects (e.g., butterflies, snails, spiders, bees) from a collection of images based on their visual features.



1.4. General Knowledge

- The number of neurons in the **output layer** is determined by the **number of classes** (since each neuron typically represents a class in classification tasks or an output value in regression).
- The number of neurons in the **input layer** is determined by the **number of inputs** (since each neuron corresponds to an input feature).

1.5. Data Representation

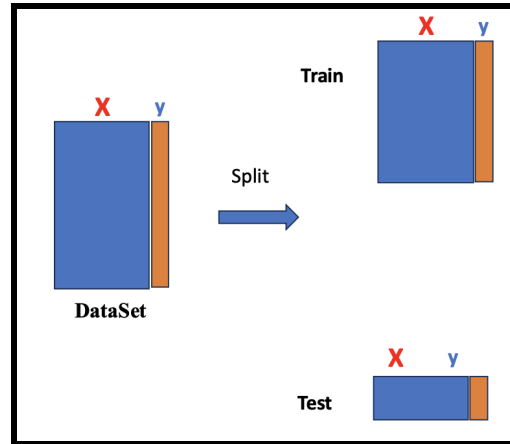
Machine learning algorithms exclusively receive numerical inputs.

- **Images:** Computers interpret images as tensors (matrices of numbers), with colored images typically having three channels (Red, Green, Blue).
- **Text:** Textual data is converted into numerical formats using embedding techniques.

2. Predictive Performance Supervised Learning

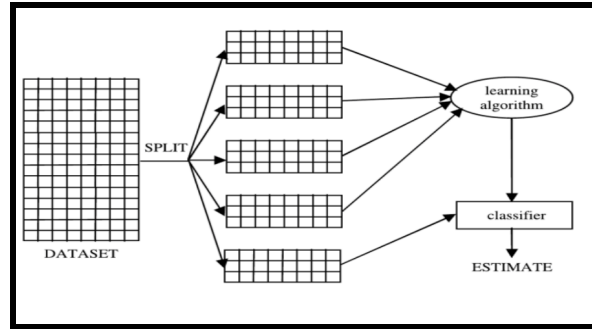
Evaluating the performance of machine learning models is crucial to ensure their effectiveness.

2.1. Data Splitting (Training and Testing Sets)



- **Concept:** The available dataset is divided into two main parts: a training set (used to train the classifier) and a testing set (used to evaluate the model). We differentiate X and y for both Train and Test by: X_{train} , y_{train} and X_{test} , y_{test} .
- **Typical Ratios:** Common splits include 70% training and 30% testing, or 80% training and 20% testing, depending on the dataset size.
- **Key Principle:** The training set should *not* be used in testing and the test set should *not* be used for training. An unseen test set provides an unbiased estimate of accuracy. The test set is also known as the **holdout set**. This method is mainly used when the dataset is large.
- **Advantages:**
 - Simple and computationally efficient.
 - Provides a quick estimate of model performance.
- **Disadvantages:**
 - The evaluation depends heavily on how the data is split, leading to high variance in performance estimates.
 - If the dataset is small, the testing set may not be representative of the overall data distribution.

2.2. K-Fold Cross-Validation



- **Concept:** The dataset is partitioned into k equal-sized disjoint subsets (folds). The model is trained and evaluated k times. In each iteration, one fold serves as the testing set, and the remaining $k-1$ folds are combined to form the training set. The final estimated performance is the average of the k performance results.
- **Purpose:** To provide a more robust estimate of model performance by averaging results across multiple splits. This method is used when the available data is not large.
- **Common Values for k :** 10-fold and 5-fold cross-validations are commonly used.
- **Advantages:**
 - Reduces the variance in performance estimates compared to a single train-test split.
 - Makes better use of the data, especially useful for small datasets.
- **Disadvantages:**
 - Computationally expensive, as the model needs to be trained and evaluated multiple times.
 - Can be time-consuming for large datasets or complex models.

2.3. Confusion Matrix

Actual / Predicted	Predicted Positive	Predicted Negative
Positive	TP (True Positive)	FN (False Negative)
Negative	FP (False Positive)	TN (True Negative)

A confusion matrix describes the breakdown of errors in predictions made by a classification model. It shows the number of correct and incorrect **predictions** compared to the **actual** outcomes (**target values**) in the data. It is an $N \times N$ matrix, where N is the number of target values (classes).

Components for Binary Classification (Positive/Negative Classes):

- **True Positive (TP):** The number of correct classifications of positive examples.
- **False Negative (FN):** The number of incorrect classifications of positive examples (model predicts negative, but actual is positive).
- **False Positive (FP):** The number of incorrect classifications of negative examples (model predicts positive, but actual is negative).
- **True Negative (TN):** The number of correct classifications of negative examples.

Derived Metrics from Confusion Matrix:

- **Accuracy:** The proportion of correct classifications from the overall number of cases.
 - $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$
- **Precision (Positive Predictive Value):** The proportion of correctly identified positive cases among all cases predicted as positive. It is the number of True Positives divided by the number of True Positives and False Positives.
 - $\text{Precision (p)} = TP / (TP + FP)$
- **Recall (Sensitivity):** The proportion of actual positive cases that were correctly identified. It is the number of True Positives divided by the number of True Positives and False Negatives.
 - $\text{Recall (r)} = TP / (TP + FN)$
- **F1-Score:** The harmonic mean of precision and recall. It combines precision and recall into one measure. For the F1-score to be large, both precision and recall must be high.
 - $F1 = 2 * (p * r) / (p + r)$
 - Alternatively: $F1 = 2 / (1/p + 1/r)$

Confusion Matrix Example (Covid-19 Prediction)

Suppose a model predicts Covid-19 status on a test set of 1000 patients, producing the following confusion matrix:

Actual \ Predicted	Covid_Positive	Covid_Negative
Covid_Positive	450 (TP)	90 (FN)
Covid_Negative	70 (FP)	390 (TN)

- **Recall (Sensitivity) for Covid_Positive:**
 - $r = TP / (TP + FN) = 450 / (450 + 90) = 450 / 540 = 0.833$
- **Precision for Covid_Positive:**
 - $p = TP / (TP + FP) = 450 / (450 + 70) = 450 / 520 = 0.865$
- **F1-Score for Covid_Positive:**

- $F1 = 2 * (0.865 * 0.833) / (0.865 + 0.833) = 2 * 0.7205 / 1.698 = 1.441 / 1.698 = 0.849$

- **Predictive Accuracy:**

- $Accuracy = (TP + TN) / (TP + TN + FP + FN) = (450 + 390) / (450 + 70 + 90 + 390) = 840 / 1000 = 0.84$

- **Specificity (for Covid_Negative, equivalent to Recall for Negative class):**

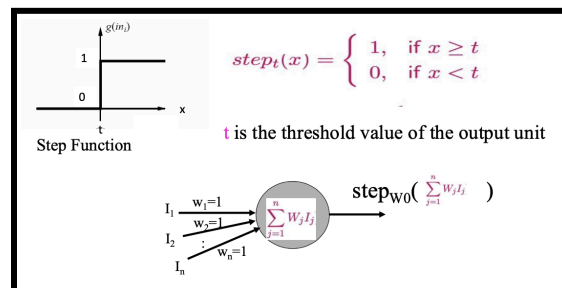
- $Specificity = TN / (TN + FP) = 390 / (390 + 70) = 390 / 460 = 0.848$

3. Neural Networks Perceptron

3.1. Perceptron

Artificial neural networks are inspired by biological brains and neurons. A neural network is a graph with nodes (units) connected by links, each with an associated weight. Each node produces an output based on a function of the weighted sum of its inputs, using an **activation function** (e.g., a step function).

- **Node/Unit:** A processing element that takes inputs, computes a weighted sum, and applies an activation function to produce an output.
- **Weights (W_j):** Real numbers associated with each link, representing the strength of the connection. They can be positive or negative and change over time during learning.
- **Threshold (t):** A value that the weighted sum must exceed for the unit to activate. The threshold can be learned like any other weight by setting $W_0 = t$ and $I_0 = -1$ (bias input). This allows assuming the unit's threshold is 0.
- **Units as Logic Gates:** Units with a threshold activation function can act as logic gates, computing Boolean functions of their inputs.
 - **AND Gate:** With inputs x_1 , x_2 , and a bias input -1 , if $w_1=1$, $w_2=1$, $w_0=1.5$, the output is 1 only when x_1 and x_2 are both 1.
 - **OR Gate:** With inputs x_1 , x_2 , and a bias input -1 , if $w_1=1$, $w_2=1$, $w_0=0.5$, the output is 1 if either x_1 or x_2 (or both) are 1.
 - **NOT Gate:** With input x_1 and a bias input -1 , if $w_1=-1$, $w_0=-0.5$, the output is 1 when x_1 is 0, and 0 when x_1 is 1.



3.2. Feed-forward Networks

These networks process information from the input layer to the output layer without internal state (only weights). They include:

- **Single-layer perceptrons:** All inputs are connected directly to the outputs. Each output unit operates independently.
- **Multi-layer perceptrons (MLPs):** Stacking different perceptrons into multiple layers to handle more complex scenarios.

3.3. Perceptron Learning (Weight Update)

The goal of learning is to adjust the weights to minimize the error between the predicted output and the target output.

- **Error Correction Method:**
 - Start with an all-zero weight vector.
 - Cycle repeatedly through the training examples. For each example:
 - If the perceptron outputs 0 but should be 1 (meaning $\text{Err} > 0$), add the input vector to the weight vector ($W_j \leftarrow W_j + \alpha * I_j * \text{Err}$, where α is the learning rate).
 - If the perceptron outputs 1 but should be 0 (meaning $\text{Err} < 0$), subtract the input vector from the weight vector.
 - Otherwise (if the output is correct), do nothing.

3.4. Example: Learning the Logical OR Function

Sample	x0	x1	x2	Label
1	1	0	0	0
2	1	0	1	1
3	1	1	0	1
4	1	1	1	1

Epoch 1: $W = \langle 0, 0, 0 \rangle$

- **Example 1 ($I = \langle 1, 0, 0 \rangle$, label=0):**
 - Perceptron: $1 \times 0 + 0 \times 0 + 0 \times 0 = 0 \rightarrow$ Output is 0
 - Correct, so W remains $\langle 0, 0, 0 \rangle$.
- **Example 2 ($I = \langle 1, 0, 1 \rangle$, label=1):**
 - Perceptron: $1 \times 0 + 0 \times 0 + 1 \times 0 = 0 \rightarrow$ Output is 0
 - Incorrect (should be 1). **Add** input to weights: $W = \langle 0, 0, 0 \rangle + \langle 1, 0, 1 \rangle = \langle 1, 0, 1 \rangle$.
- **Example 3 ($I = \langle 1, 1, 0 \rangle$, label=1):**
 - Perceptron: $1 \times 1 + 1 \times 0 + 0 \times 1 = 1 \rightarrow$ Output is 1
 - With $W = \langle 1, 0, 1 \rangle$, output is 1. Correct, so W remains $\langle 1, 0, 1 \rangle$.
- **Example 4 ($I = \langle 1, 1, 1 \rangle$, label=1):**

- Perceptron: $1 \times 1 + 1 \times 0 + 1 \times 1 = 1 \rightarrow$ Output is 1
- With $W = \langle 1, 0, 1 \rangle$, output is 1. Correct, so W remains $\langle 1, 0, 1 \rangle$.

Epoch 2: $W = \langle 1, 0, 1 \rangle$

- **Example 1 ($I = \langle 1, 0, 0 \rangle$, label=0):**
 - Perceptron: $1 \times 1 + 0 \times 0 + 0 \times 1 = 1 \rightarrow$ Output is 1
 - Incorrect (should be 0). **Subtract** input to weights: $W = \langle 1, 0, 1 \rangle - \langle 1, 0, 0 \rangle = \langle 0, 0, 1 \rangle$.
- **Example 2 ($I = \langle 1, 0, 1 \rangle$, label=1):**
 - Perceptron: $1 \times 0 + 0 \times 0 + 1 \times 1 = 1 \rightarrow$ Output is 1
 - With $W = \langle 0, 0, 1 \rangle$, output is 1. Correct, so W remains $\langle 0, 0, 1 \rangle$.
- **Example 3 ($I = \langle 1, 1, 0 \rangle$, label=1):**
 - Perceptron: $1 \times 0 + 1 \times 0 + 0 \times 1 = 0 \rightarrow$ Output is 0
 - Incorrect (should be 1). **Add** input to weights: $W = \langle 0, 0, 1 \rangle + \langle 1, 1, 0 \rangle = \langle 1, 1, 1 \rangle$.
- **Example 4 ($I = \langle 1, 1, 1 \rangle$, label=1):**
 - Perceptron: $1 \times 1 + 1 \times 1 + 1 \times 1 = 1 \rightarrow$ Output is 1
 - With $W = \langle 1, 1, 1 \rangle$, output is 1. Correct, so W remains $\langle 1, 1, 1 \rangle$.

Epoch 3: $W = \langle 1, 1, 1 \rangle$

- **Example 1 ($I = \langle 1, 0, 0 \rangle$, label=0):**
 - Perceptron: $1 \times 1 + 0 \times 1 + 0 \times 1 = 1 \rightarrow$ Output is 1
 - Incorrect (should be 0). **Subtract** input to weights: $W = \langle 1, 1, 1 \rangle - \langle 1, 0, 0 \rangle = \langle 0, 1, 1 \rangle$.
- **Example 2 ($I = \langle 1, 0, 1 \rangle$, label=1):**
 - Perceptron: $1 \times 0 + 0 \times 1 + 1 \times 1 = 1 \rightarrow$ Output is 1
 - With $W = \langle 0, 1, 1 \rangle$, output is 1. Correct, so W remains $\langle 0, 1, 1 \rangle$.
- **Example 3 ($I = \langle 1, 1, 0 \rangle$, label=1):**
 - Perceptron: $1 \times 0 + 1 \times 1 + 0 \times 1 = 1 \rightarrow$ Output is 1
 - With $W = \langle 0, 1, 1 \rangle$, output is 1. Correct, so W remains $\langle 0, 1, 1 \rangle$.
- **Example 4 ($I = \langle 1, 1, 1 \rangle$, label=1):**
 - Perceptron: $1 \times 0 + 1 \times 1 + 1 \times 1 = 1 \rightarrow$ Output is 1
 - With $W = \langle 0, 1, 1 \rangle$, output is 1. Correct, so W remains $\langle 0, 1, 1 \rangle$.

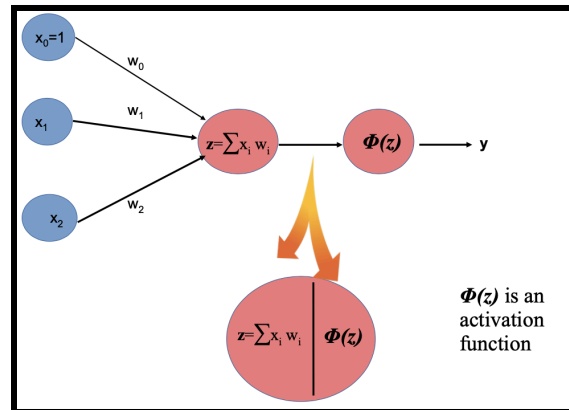
Epoch 4: $W = \langle 0, 1, 1 \rangle$

- **Example 1 ($I = \langle 1, 0, 0 \rangle$, label=0):**
 - Perceptron: $1 \times 0 + 0 \times 1 + 0 \times 1 = 0 \rightarrow$ Output is 0
 - With $W = \langle 0, 1, 1 \rangle$, output is 0. Correct, so W remains $\langle 0, 1, 1 \rangle$.
- All remaining examples will also be correctly classified. The perceptron has learned the OR function with final weights $W = \langle 0, 1, 1 \rangle$.

Lecture 4: Multi-layer Perceptron (MLP)

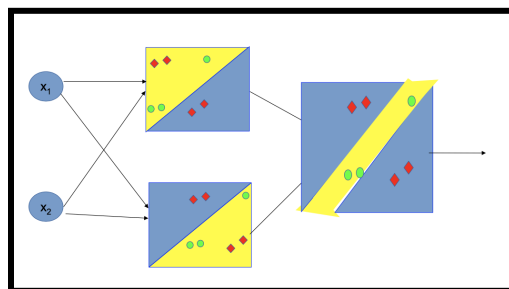
This lecture introduces the concept of Multi-layer Perceptrons (MLP) and various activation functions and their applications in classification problems.

4.1. Perceptron Basics:



- A perceptron takes multiple inputs, multiplies them by weights, sums them up, and passes the result through an activation function to produce an output.
- It classifies binary linearly separable data effectively.
- **Limitations** arise when data is not linearly separable, as a single line cannot separate the classes.

4.2. Layered Perceptrons (MLP):



- MLPs overcome the limitations of single perceptrons by stacking multiple perceptrons into different layers.
- Multi-class classification can be achieved using several output perceptrons.
- Nonlinear activation functions are crucial for handling complex data, as linear activation functions (step functions) are insufficient for complex scenarios.

4.3. Activation Functions:

- **Sigmoid Function:**

- Converts points into a probability of belonging to a certain class (output range 0 to 1).
- Commonly used in feedforward neural networks that need to output only positive numbers.
- Example provided for classifying overweight individuals based on height and weight, where a sigmoid output above 0.5 predicts overweight.

$$z = w_0 + w_1 * 72 + w_2 * 190 = -3.95 + 0.15 * 72 + 0.05 * 190 = 2.65$$

$$p(y = 1 | x) = 1 / (1 + \exp(-z)) = 1 / (1 + \exp(-2.65)) = 0.93$$

The predicted probability of being overweight for this individual is 0.93, which is above the threshold of 0.5. Therefore, we would classify this individual as overweight.

- **Hyperbolic Tangent Function (tanh):**
 - Formula:
 - Output ranges from -1 to 1.
- **Rectified Linear Unit (ReLU):**
 - Currently recommended in research due to superior training results.
 - Most neural networks should use ReLU in hidden layers.
- **SoftMax (Normalized Exponential) Activation Function:**
 - A generalization of the sigmoid function for multi-class classification.
 - Transforms a K-dimensional vector of arbitrary real values into a K-dimensional vector of real values, where each entry is in the range (0, 1) and all entries sum up to 1.
 - Calculates the probabilities of each target class over all possible target classes.
 - Typically found in the output layer of a classification neural network.
 - The neuron with the highest value claims the input as a member of its class, representing the probability that the input falls into each class.
 - Example demonstrates how Softmax units represent a probability distribution over discrete variables.

•probability 1 = $\exp(1) / (\exp(1) + \exp(3) + \exp(2)) = 0.09003057317038046$
 •probability 3 = $\exp(3) / (\exp(1) + \exp(3) + \exp(2)) = 0.6652409557748219$
 •probability 2 = $\exp(2) / (\exp(1) + \exp(3) + \exp(2)) = 0.24472847105479767$

- **Formulas:**

Sigmoid Function	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent Function (tanh)	$\tanh(X) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Rectified Linear Unit (ReLU)	$ReLU(X) = \max(0, x)$
SoftMax	$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$ For $j=1, 2, \dots, k$

4.4. Network Classification (Binary vs. Multi-class):

- **Classification:** The process where a neural network categorizes input into one or more classes.
- **Binary Classification:** Classifies input into one of two classes (e.g., "Is this a '2' or not?").
- **Multi-class Classification:** Classifies input into more than two classes (e.g., "Which digit is this?").

4.5. Evaluating the Network:

- A **Cost/Error/Loss Function** evaluates how well the predicted output aligns with the actual output.
- The goal during training is to minimize this loss function to improve predictive accuracy.
- **Empirical Error Optimization:** Involves updating weights to minimize the loss function.

- The problem is a function minimization problem, where the empirical error with respect to weights over the data samples is minimized.

4.6. Representing Input and Output:

- **Input:** Typically a vector of numbers (e.g., pixel values, speech features, real-valued vectors representing text).
- **Output (Real-valued):**
 - Scalar Output: Single output neuron for a single real value.
 - Vector Output: Multiple output neurons, one for each dimension of the desired output.
- **Output (Binary):**
 - Simple 1/0 representation (e.g., 1 for "yes it's a cat," 0 for "no it's not a cat").
 - Sigmoid activation is typically used for binary output, representing the probability of class value 1.
 - Can also be represented by two independent outputs: one for the desired output and one for its negation (e.g., [1 0] for Yes, [0 1] for No).
- **Output (Multi-class - One-hot Representation):**
 - For N classes, the one-hot representation has N binary outputs, an N-dimensional binary vector.
 - The neural network's output ideally has N-1 zeros and a single 1 in the correct position; realistically, it's a probability vector with N probability values that sum to 1.
 - Softmax activation is commonly used in the output layer for multi-class classifiers.

4.7. Loss Functions:

- **L2 Loss (Regression):**

$$loss(y', y) = \frac{1}{2} \|y' - y\|^2 = \frac{1}{2} \sum_i (y'_i - y_i)^2$$

- Used for real-valued output vectors.
 - Differentiable.
- **Cross-Entropy Loss (Binary Classifier):**

$$loss(y', y) = -y \log(y') - (1 - y) \log(1 - y')$$

- Used for binary classification with a single output (y is 0 or 1, and $\hat{y}$ is between 0 and 1).
- Minimum when $\hat{y} = y$.
- **Cross-Entropy Loss (Multi-Class Classification):**

$$loss(y', y) = - \sum_i y_i \log(y'_i)$$

- Desired output y is a one-hot vector.
- Actual output will be a probability distribution.
- The goal is to minimize this loss function during training.

Lecture 5: Multi-layer Perceptron (MLP) Continued

This lecture continues the discussion on Multi-layer Perceptrons (MLP), focusing on optimization algorithms for training neural networks, particularly different forms of Gradient Descent.

5.1. Gradient Descent: Recap

- The training process of a neural network involves feeding data, making predictions, calculating loss (e.g., Categorical Cross-entropy), and using this loss to update the model parameters.
- Training neural networks on large datasets can be slow, even though deep learning does not perform best in a big data regime. Therefore, fast optimization algorithms are essential to speed up model efficiency.

5.2. Batch Gradient Descent

- Batch Gradient Descent processes the entire training set at once to calculate the gradient and update weights.
- **Disadvantage:** The main drawback is that it takes a long time per iteration when processing a huge training set.

5.3. Mini-Batches

- Mini-batches are used to speed up the gradient descent process by processing smaller subsets of the training data.
 - **Total number of training examples (N):** The total instances in the dataset.
 - **Batch size (B):** The number of training examples processed in one forward/backward pass.
 - **Number of Batches (t):** Calculated as $t = \lceil N/B \rceil$ (rounded to the nearest integer).
 - **Epoch:** One full pass through the entire training dataset. For example, if there are 79 batches, one epoch involves processing all 79 batches.
- **Mini-Batch Gradient Descent:** Mini-batch gradient descent processes one mini-batch at a time, rather than the entire training set. For each mini-batch, a single step of gradient descent is implemented.

5.4. Size of Mini-Batch (Parameter):

- If the mini-batch size $B = N$ (the entire data), it's called **Batch Gradient Descent**.
- If the mini-batch size $B = 1$, it's called **Stochastic Gradient Descent (SGD)**, where every example is its own mini-batch.
- **Stochastic Gradient Descent:** Can be very noisy, and while it generally moves the gradient in a good direction, it can sometimes head in the wrong direction. A small learning rate can reduce noisiness, but a significant disadvantage is the loss of vectorization speed.

- **Mini-batch Size:** In practice, a size that is "not too big, not too small" works best, providing the fastest learning. It sits between Batch Gradient Descent and Stochastic Gradient Descent.

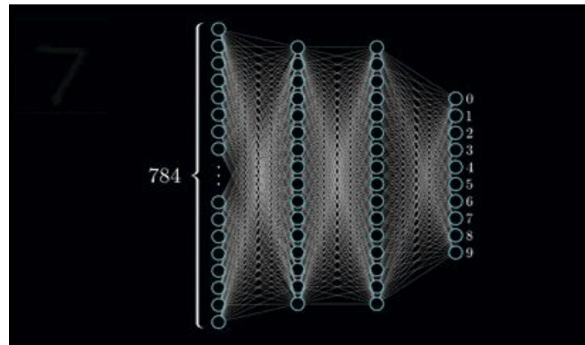
5.5. Choosing Mini-Batch Size

- For small training sets, batch gradient descent works well.
- For larger training sets, typical mini-batch sizes range from 64 to 512.
- Empirical studies suggest that mini-batch processing becomes faster when the batch size is a power of 2 (e.g., 64, 128, 256, 512), as this allows mini-batches to fit efficiently into CPU/GPU memory.

5.6. Other Optimization Algorithms

- There are alternatives to standard gradient descent that aim to speed up convergence by choosing different search directions or step sizes. Examples include:
 - Adam
 - SGD
 - RMSprop

5.7. Feeding Images to Feed Forward Neural Networks



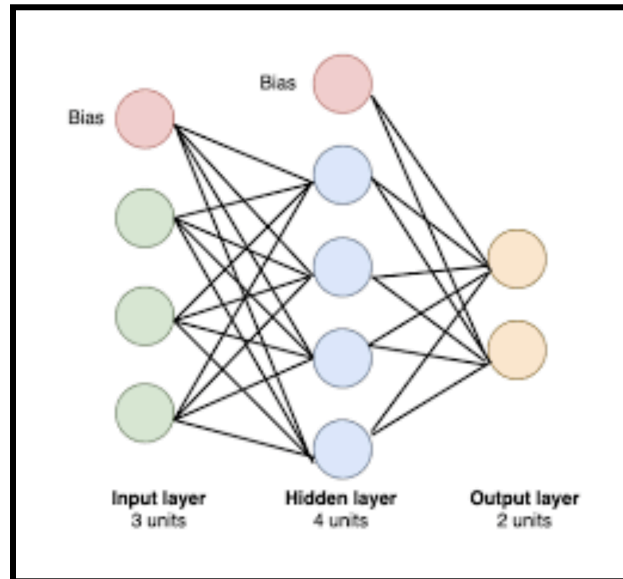
- Images, such as a 28x28 image, are typically flattened into a single vector (e.g., 784 dimensions for a grayscale image) before being fed into a feed-forward neural network.

Lecture 6: Learning Neural Networks

This lecture focuses on the foundational concepts of learning in neural networks, including parameter calculation, the training process, different loss functions, and optimization techniques.

6.1. Parameters of a Neural Network

- The number of parameters in a neural network depends on the number of weights and layers.



- **Calculation:** For any given layer, the total number of parameters is calculated as:
$$\text{Parameters} = (\text{in} \times \text{out}) + \text{number of Biases}$$
 - **In:** Number of input neurons to the layer.
 - **Out:** Number of output neurons from the layer.
 - **Number of Biases:** Equal to the number of output neurons.
- **Goal of Learning:** Learning in a neural network means learning the optimal values of all these parameters (weights) in each layer.

6.2. The Training Process

- The training process of a neural network involves several key steps:
- **Forward Propagation:** This is the process where inputs are passed through the network to generate predictions.
- **Loss Function:** Quantifies the difference between the network's predictions and the true values. Examples include Mean Squared Error and Cross-Entropy Loss. The objective is to minimize this loss function during training to improve predictive accuracy.
- **Backpropagation:** This mechanism computes the gradients of the loss function with respect to each weight, utilizing the chain rule of calculus.

- **Gradient Descent (GD):** Weights are iteratively updated to minimize the loss function.

6.3. Gradient Descent (GD) Multi-variables

- For each weight (W_k), Gradient Descent involves:
 1. Initializing weights and biases to small random values.
 2. Calculating the cost function for the entire dataset.
 3. Calculating the gradient of the cost function with respect to each weight (W_k).
 4. Updating each weight using the formula: $W_k = W_k - \alpha(\partial J / \partial W_k)$, where α is the learning rate.
 5. Repeating these steps until the cost function converges.

6.4. Binary Cross-Entropy and Multi-Class Classification: Loss Cross-Entropy

- **Binary cross-entropy** is a loss function used in binary classification tasks where the target values are 0 or 1. It measures how well the model's predicted probabilities match the actual labels. The model aims to minimize this loss during training to improve accuracy.
- **Desired Output (y):** A one-hot vector (e.g., [0 0...1...0 0 0]) with '1' at the position of the correct class 'c'.
- **Actual Output (p):** A probability distribution (e.g., [p1,p2,...]).
- **Cross-Entropy Loss Formula:** $\text{Loss}(y,p)$, where this formula quantifies the discrepancy between the predicted probability distribution and the true distribution.

$$\text{Loss}(y,p) = - \sum_i y_i \log(p_i)$$

6.5. Regression: Loss Function

- For regression tasks, where the output is a continuous value, common loss functions include:
 - **Mean Absolute Error (MAE):**

$$\text{MAE}(y,p) = \frac{1}{N} \sum_1^N |y_i - p_i|$$

- **Mean Squared Error (MSE):**

$$\text{MAE}(y,p) = \frac{1}{N} \sum_1^N (y_i - p_i)^2$$

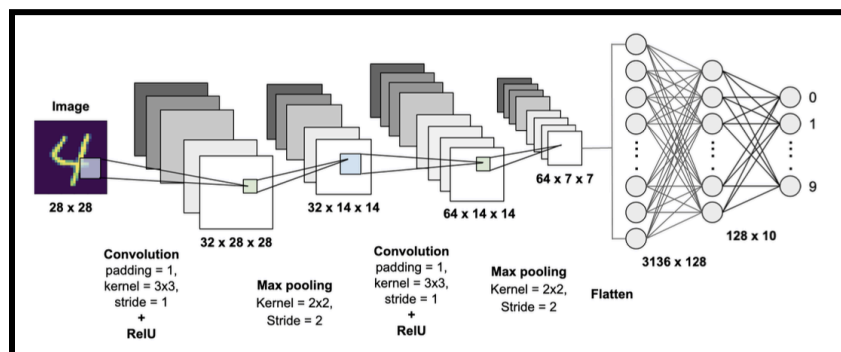
Lecture 7: Convolutional Neural Network (CNN) Notes

This lecture introduces Convolutional Neural Networks (CNNs) as a specialized type of neural network particularly effective for image analysis, addressing limitations of traditional Multi-Layer Perceptrons (MLPs) when dealing with image data.

7.1. Limitations of Multi-Layer Perceptrons (MLPs) for Image Data

When feeding images to traditional MLPs, images (e.g., a 28x28 grayscale image) must be flattened into a single vector (e.g., 784 dimensions). For RGB images (28x28x3), this results in an even larger vector (2352 dimensions). This flattening process disregards the spatial relationships between pixels, which are crucial for image understanding. Additionally, fully connected models in MLPs imply a massive number of edges (weights), and questions arise whether all these connections are necessary or if weights can be shared, and how the position of features (e.g., a cat's face in an image) is handled if the pattern appears in different places.

7.2. Introduction to Convolutional Neural Networks (CNNs)



CNNs are artificial neural networks specifically designed for pattern detection in images. This pattern detection capability makes CNNs highly effective for image analysis. CNNs utilize hidden layers called "convolutional layers" which are fundamental to their effectiveness.

7.3. Filters/Kernels and Pattern Detection

Each convolutional layer requires a specified number of *filters* or *kernels*. These filters are responsible for detecting specific patterns within the image, such as:

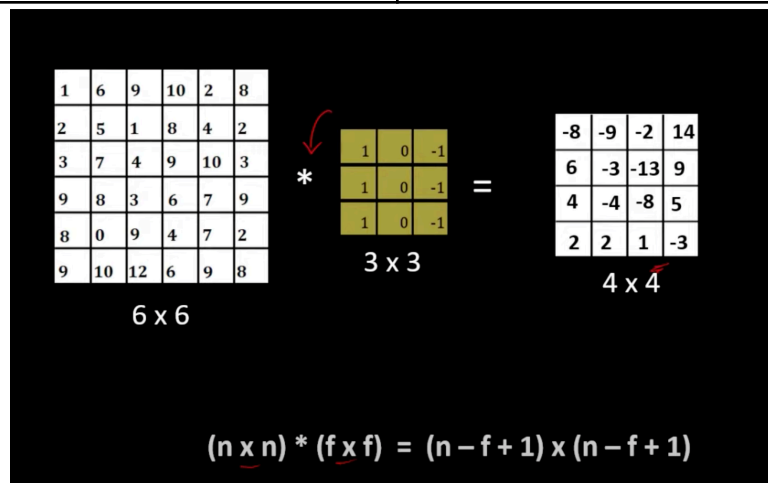
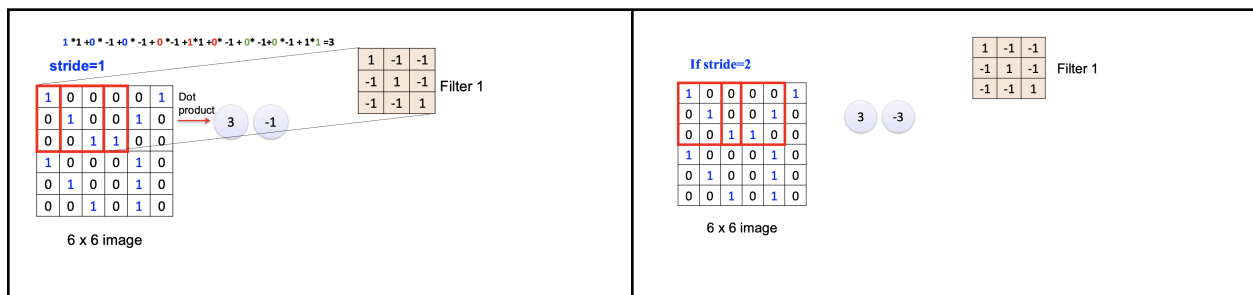
- Edges
- Shapes
- Textures
- Curves
- Objects
- Colors

For example, a filter designed to detect edges is known as an "edge detector".

7.4. Convolutional Layer and Operation

A CNN includes convolutional layers that perform convolutional operations. A filter *convolves* by sliding across blocks of pixels from the input image. The convolution process involves an input image (blue), a filter (shaded 3x3), and produces an output (green). The filters themselves contain parameters that are learned during network training. Each filter detects a small pattern.

- **Convolution Steps:** The operation involves performing a dot product between the filter and a region of the input image.
- **Stride:** The step size by which the filter moves across the image. A **stride=1** means the filter moves one pixel at a time. A **stride=2** means the filter moves two pixels at a time.



- **Padding:** It is important because we can end up losing the value of the analysis of the image by reducing the dimensions too much, so we can use zero padding to focus the filters more on the center, which is the original matrix.
 - **No Padding (Valid Padding):** If an input image of size $n \times n$ is convolved with an $f \times f$ filter, the output size is $(n-f+1) \times (n-f+1)$. This reduces the dimensions of the output.
 - **Zero Padding (Same Padding):** Adding zeros around the image allows the dimensions to remain the same after convolution.

7.5. Handling Color Images (RGB)

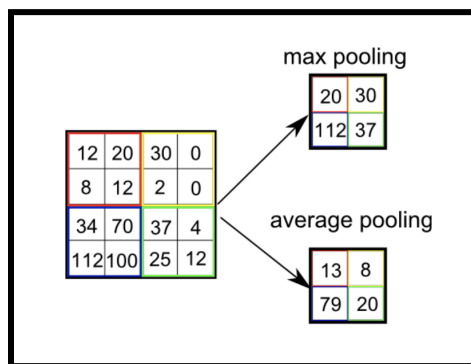
Color images typically have 3 channels (Red, Green, Blue). A convolutional operation on an RGB image involves applying filters across these multiple channels.

7.6. Convolution vs. Fully Connected Networks

CNNs offer advantages over fully connected networks by:

- **Reducing the number of connections:** Not all inputs are connected to every neuron in the next layer.
- **Sharing weights:** The same filter (weights) is applied across different parts of the input image, leading to fewer parameters. This significantly reduces the number of parameters compared to a fully connected network, making CNNs more efficient for image processing.

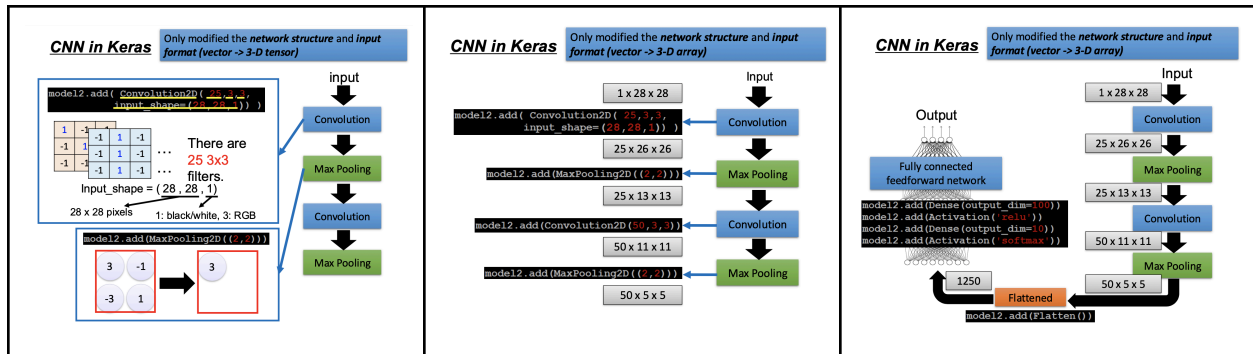
7.7. Pooling



Pooling layers are used to further reduce the complexity of the network and condense information.

- **Purpose:** Subsampling pixels within the image does not change the essential features of an object (e.g., a bird) and helps reduce the image size, requiring fewer parameters to characterize it.
- **Types of Pooling:**
 - **Max Pooling:** Extracts the maximum value from a defined window (e.g., 2x2).
 - **Average Pooling:** Calculates the average value within a defined window.
- **Effect:** Pooling further reduces the complexity of a CNN by compressing the feature maps.

7.8. CNN Implementation in Keras



Implementing CNNs in Keras involves defining the network structure with specific layers for convolutional operations and pooling.

- **Convolutional Layer:** `Convolution2D` (or `Conv2D`) is used, specifying the number of filters, kernel size (e.g., (3,3)), and input shape (e.g., (28,28,1) for grayscale images, or (28,28,3) for RGB images).
- **Pooling Layer:** `MaxPooling2D` is used to apply max pooling (e.g., (2,2) pool size).
- **Flattening:** After several convolutional and pooling layers, the output is often flattened using `Flatten()` to feed it into a fully connected `Dense` layer for classification.
- **Padding in Keras:** Keras allows specifying `padding='valid'` (no padding) or `padding='same'` (zero padding) in `Conv2D` layers. Valid padding reduces dimensions, while same padding maintains them.

7.9. Popular CNN Architectures

The lecture mentions several prominent CNN architectures that have won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC):

- **AlexNet (2012):** A relatively early but influential CNN with 8 layers and 60 million parameters.
- **VGGNet (2014):** Known for its simplicity and depth, typically using 3x3 convolutions throughout. VGG16 has 16 layers and 138 million parameters.
- **ResNet (2015):** Introduced residual connections to enable training very deep networks (e.g., 152 layers in ResNet).

7.10. Transfer Learning

Transfer learning is a powerful technique that involves using a CNN pre-trained on a very large dataset (like ImageNet) as a starting point, instead of training a model from scratch.

- **Representation Extraction:** This involves keeping the convolutional base (which acts as a feature extractor) of the pre-trained model frozen and then feeding the extracted features (bottleneck features) into a new classifier (often a few dense layers) for the

specific task. Earlier layers learn generic features (edges, colors, textures), while later layers learn more abstract concepts (like a dog's ear).

- **Fine-tuning:** Instead of freezing the entire convolutional base, fine-tuning involves unfreezing and retraining some of the later layers of the pre-trained model to particularize it to the new task. Care must be taken not to have large gradient updates during fine-tuning.